

Report FIT3003 Assignment 2

Name: Ng Li Xian
Student ID: 33986010
Date: 17 October 2025

Table of Contents

Transformation Stage	3
B. Preparation Stage: Data Cleaning Strategies	3
1. Handling Null Value Problems	3
2. Removing Duplication Problems	5
3. Resolving Inconsistent Values	7
4. Fixing Relationship Problems (Referential Integrity)	9
5. Correcting Incorrect Values	11
C. Snowflake/ Star Schema (design task A)	15
F. Screenshots of table structure (design task A)	15
D. List suggestion of increase the granularity of fact tables (design task B)	19
E. Explanation for each SCD Types and Temporal Dimension	20
Data Analytic Stage	22
G. Findings Report	22
References	26

Transformation Stage

B. Preparation Stage: Data Cleaning Strategies

1. Handling Null Value Problems

In the **CATEGORY** table,

Before

For CATEGORY_ID = 15, the CATEGORY_DESCRIPTION is null.

CATEGORY_ID	CATEGORY_DESCRIPTION
1	Access
2	Air Compressor
3	Compaction
4	Concrete
5	Earthmoving
6	Generators
7	Landscaping
8	Lighting
9	Plumbing
10	Rail
11	Safety
12	Site Equipment
13	Trailers
14	Vehicles
15	null

CATEGORY_ID	CATEGORY_DESCRIPTION
1	15 null

SQL Code

```
select *
from MonEquip.CATEGORY
where CATEGORY_ID is null
   or trim(CATEGORY_ID) = ''
   or upper(trim(CATEGORY_ID)) = 'NULL'
   or CATEGORY_DESCRIPTION is null
   or trim(CATEGORY_DESCRIPTION) = ''
   or upper(trim(CATEGORY_DESCRIPTION)) = 'NULL';
```

After

The null value in the CATEGORY_DESCRIPTION is removed with DELETE

CATEGORY_ID	CATEGORY_DESCRIPTION
1	Access
2	Air Compressor
3	Compaction
4	Concrete
5	Earthmoving
6	Generators
7	Landscaping
8	Lighting
9	Plumbing
10	Rail
11	Safety
12	Site Equipment
13	Trailers
14	Vehicles

CATEGORY_ID	CATEGORY_DESCRIPTION
-------------	----------------------

SQL Code

```
CREATE TABLE CATEGORY AS
SELECT *
from MonEquip.CATEGORY;

delete from CATEGORY
where CATEGORY_ID is null
   or trim(CATEGORY_ID) = ''
   or upper(trim(CATEGORY_ID)) = 'NULL'
   or CATEGORY_DESCRIPTION is null
   or trim(CATEGORY_DESCRIPTION) = ''
   or upper(trim(CATEGORY_DESCRIPTION)) = 'NULL';
-- check
select *
from CATEGORY
where CATEGORY_ID is null
   or trim(CATEGORY_ID) = ''
   or upper(trim(CATEGORY_ID)) = 'NULL'
   or CATEGORY_DESCRIPTION is null
   or trim(CATEGORY_DESCRIPTION) = ''
   or upper(trim(CATEGORY_DESCRIPTION)) = 'NULL';
select * from CATEGORY;
-- initial
select * from MonEquip.CATEGORY;
```

2. Removing Duplication Problems

- a. In the **CUSTOMER** table,

Before

There is duplication of the CUSTOMER_ID = 52

	CUSTOMER_ID	COUNT(*)
1	52	4

	CUSTOMER_TYPE_ID	NAME	GENDER	ADDRESS_ID	PHONE	EMAIL	COUNT(*)
1	2	Abbie Maddie	Male		52 904 627 9038	amaddie1@columbia.edu	4

SQL Code

```
select CUSTOMER_ID, COUNT(*)
from MonEquip.CUSTOMER
group by CUSTOMER_ID
having count(*) > 1;

select CUSTOMER_TYPE_ID, NAME, GENDER, ADDRESS_ID, PHONE,
EMAIL, COUNT(*)
from MonEquip.CUSTOMER
group by CUSTOMER_TYPE_ID, NAME, GENDER, ADDRESS_ID, PHONE,
EMAIL
having count(*) > 1;
```

After

The duplication of the CUSTOMER_ID = 52 is removed with DISTINCT

	CUSTOMER_ID	COUNT(*)
--	-------------	----------

SQL Code

```
CREATE TABLE CUSTOMER AS
SELECT DISTINCT *
FROM MonEquip.CUSTOMER;
-- check
select CUSTOMER_ID, COUNT(*)
from CUSTOMER
group by CUSTOMER_ID
having count(*) > 1;
SELECT * FROM CUSTOMER;
-- initial
select * from MonEquip.CUSTOMER;
```

- b. In the **CUSTOMER_TYPE** table,

Before

There is duplication of the description 'Business'

	CUSTOMER_TYPE_ID		DESCRIPTION
1		1	Individual
2		2	Business
3		2	business

	CUSTOMER_TYPE_ID		COUNT(*)
1		2	2

SQL Code

```
select CUSTOMER_TYPE_ID, COUNT(*)
from MonEquip.CUSTOMER_TYPE
group by CUSTOMER_TYPE_ID
having count(*) > 1;
```

After

The duplication of description 'Business' is removed with GROUP BY

	CUSTOMER_TYPE_ID		DESCRIPTION
1		1	Individual
2		2	Business

SQL Code

```
CREATE TABLE CUSTOMER_TYPE AS
SELECT CUSTOMER_TYPE_ID,
       INITCAP(LOWER(DESCRIPTION)) AS DESCRIPTION
FROM MonEquip.CUSTOMER_TYPE
GROUP BY CUSTOMER_TYPE_ID, INITCAP(LOWER(DESCRIPTION));
-- check
SELECT * FROM CUSTOMER_TYPE;
-- initial
select * from MonEquip.CUSTOMER_TYPE;
```

3. Resolving Inconsistent Values

- a. In **CUSTOMER** table,

Before

Inconsistent values of Male and Female. There is M and F.

	CUSTOMER_ID	CUSTOMER_TYPE_ID	NAME	GENDER	ADDRESS_ID
1	27	2	Darrelle Durgan	F	27
2	79	1	Swen Huddlestone	M	79
3	87	2	Wilma Abyss	F	87
4	139	1	Ned Halliberton	M	139

SQL Code

```
select *  
from MonEquip.CUSTOMER  
where GENDER not in ('Male', 'Female');
```

After

The inconsistent values of Male and Female is resolved with CASE

	CUSTOMER_ID	CUSTOMER_TYPE_ID	NAME	GENDER	ADDRESS_ID	PHONE
--	-------------	------------------	------	--------	------------	-------

SQL Code

```
UPDATE CUSTOMER  
SET GENDER = CASE  
    WHEN UPPER(GENDER) IN ('M', 'MALE') THEN 'Male'  
    WHEN UPPER(GENDER) IN ('F', 'FEMALE') THEN 'Female'  
    ELSE 'Other'  
END;  
-- check  
select *  
from CUSTOMER  
where GENDER not in ('Male', 'Female');  
SELECT * FROM CUSTOMER;  
-- initial  
select * from MonEquip.CUSTOMER;
```

- b. In **EQUIPMENT** table,

Before

The CATEGORY_ID refers to the null category being removed.

Therefore, when cleaning the inconsistent name of HITACHI manufacturer, the CATEGORY_ID is also updated to 12 (Site Equipment)

	EQUIPMENT_ID	MANUFACTURER	CATEGORY_ID
1	2	Hitachi	1
2	11	Hitachi	1
3	15	Hitachi	2
4	20	Hitachi	2
5	27	Hitachi	3
6	48	Hitachi	5
7	52	Hitachi	5
8	67	Hitachi	6
9	90	Hitachi	8
10	146	Hitachi	13
11	154	Hitachi	14
12	158	HITACHI	15

SQL Code

```
select distinct MANUFACTURER
from MonEquip.EQUIPMENT
order by MANUFACTURER;

select EQUIPMENT_ID, MANUFACTURER, CATEGORY_ID
from MonEquip.EQUIPMENT
where upper(trim(MANUFACTURER)) = 'HITACHI';
```

After

The inconsistent name of HITACHI manufacturer is now uniform

	EQUIPMENT_ID	MANUFACTURER	CATEGORY_ID
1	158	Hitachi	12

SQL Code

```
CREATE TABLE EQUIPMENT AS
SELECT *
FROM MonEquip.Equipment;

UPDATE EQUIPMENT
SET MANUFACTURER = 'Hitachi', CATEGORY_ID = 12
WHERE EQUIPMENT_ID = 158;

-- check
SELECT EQUIPMENT_ID, MANUFACTURER, CATEGORY_ID
FROM EQUIPMENT
```

```
WHERE EQUIPMENT_ID = 158;
select * from EQUIPMENT;
-- initial
select * from MonEquip.EQUIPMENT;
```

4. Fixing Relationship Problems (Referential Integrity)

In **HIRE** table,

Before

The table HIRE have breach of referential integrity with EQUIPMENT, CUSTOMER and STAFF table

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_P
1	301	08/12/20	08/12/20	190	1	

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_P
1	301	08/12/20	08/12/20	190	1	

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_P
1	301	08/12/20	08/12/20	190	1	
2	303	25/01/90	27/12/99	43	3	
3	302	05/12/20	17/10/20	21	2	
4	304	08/12/20	08/12/20	114	1	

SQL Code

```
select *
from MonEquip.HIRE
where EQUIPMENT_ID not in
(select EQUIPMENT_ID
from MonEquip.EQUIPMENT); -- invalid FK values

select *
from MonEquip.HIRE
where CUSTOMER_ID not in
(select CUSTOMER_ID
from MonEquip.CUSTOMER); -- invalid FK values

select *
from MonEquip.HIRE
where STAFF_ID not in
(select STAFF_ID
from MonEquip.STAFF); -- invalid FK values
```

After

Breach of referential integrity is resolved

(Output for all three select statements for checking)

HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_P
---------	------------	----------	--------------	----------	-------------

SQL Code

```
CREATE TABLE HIRE AS
SELECT
    H.HIRE_ID,
    H.START_DATE,
    H.END_DATE,
    H.EQUIPMENT_ID,
    H.QUANTITY,
    H.UNIT_HIRE_PRICE,
    H.TOTAL_HIRE_PRICE,
    H.CUSTOMER_ID,
    H.STAFF_ID
FROM MonEquip.HIRE H
JOIN EQUIPMENT E ON H.EQUIPMENT_ID = E.EQUIPMENT_ID
JOIN CUSTOMER C ON H.CUSTOMER_ID = C.CUSTOMER_ID
JOIN STAFF S ON H.STAFF_ID = S.STAFF_ID
WHERE h.START_DATE >= DATE '2018-04-01'
AND h.END_DATE <= DATE '2020-12-31'
AND h.END_DATE >= h.START_DATE;
-- check
select *
from HIRE
where EQUIPMENT_ID not in
(select EQUIPMENT_ID
from EQUIPMENT); -- invalid FK values

select *
from HIRE
where CUSTOMER_ID not in
(select CUSTOMER_ID
from CUSTOMER); -- invalid FK values

select *
from HIRE
where STAFF_ID not in
(select STAFF_ID
from STAFF); -- invalid FK values
```

```
select * from HIRE;
-- initial
select * from MonEquip.HIRE;
```

5. Correcting Incorrect Values

a. In **HIRE** table,

Before

There is:

Negative TOTAL_HIRE_PRICE

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	U
1	303	25/01/90	27/12/99	43	3	
2	304	08/12/20	08/12/20	114	1	

Revenue formula mismatch

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE
1	1	11/05/18	14/05/18	135	3	
2	2	17/05/18	20/05/18	49	2	
3	4	21/05/18	25/05/18	36	1	
4	5	21/05/18	23/05/18	37	2	
5	6	22/05/18	26/05/18	53	2	
6	7	24/05/18	28/05/18	73	3	
7	8	25/05/18	28/05/18	135	3	
8	11	12/06/18	14/06/18	71	2	
9	18	12/07/18	14/07/18	25	3	

Invalid date

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY
1	302	05/12/20	17/10/20		21

Invalid date (not within transaction period)

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_P
1	303	25/01/90	27/12/99	43	3	

SQL Code

```
-- negative total_hire_price
select *
from MonEquip.Hire
where UNIT_HIRE_PRICE <= 0 OR TOTAL_HIRE_PRICE < 0;
-- revenue formula mismatch
SELECT *
from MonEquip.HIRE
```



```

        END;
-- check
---- negative total_hire_price
select *
from Hire
where UNIT_HIRE_PRICE <= 0 OR TOTAL_HIRE_PRICE < 0;
-- revenue formula mismatch
SELECT *
from HIRE
WHERE TOTAL_HIRE_PRICE !=
CASE
    WHEN END_DATE = START_DATE THEN 0.5 *
UNIT_HIRE_PRICE * QUANTITY
    ELSE (END_DATE - START_DATE) * UNIT_HIRE_PRICE *
QUANTITY
END;
-- invalid date
select *
from HIRE
where End_Date < Start_Date;
-- invalid date (not within transaction period)
select *
from HIRE
WHERE START_DATE < DATE '2018-04-01' OR END_DATE > DATE
'2020-12-31';
select * from HIRE;
-- initial
select * from MonEquip.HIRE;

```

b. In **SALES** table,

Before

There is:

Quantity issue

	SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE	TOTAL_SALES_PRICE
1	151	15/12/20	20	-3	45500	

SQL Developer

Revenue Mismatch

	SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE	TOTAL_SALES_PRICE
1	151	15/12/20	20	-3	45500	

SQL Code

```
-- quantity issue
SELECT *
from MonEquip.SALES
WHERE Quantity <= 0;

-- revenue mismatch
SELECT *
from MonEquip.SALES
WHERE Total_Sales_Price != Unit_Sales_Price * Quantity;
```

After

	SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE

SQL Code

```
CREATE TABLE SALES AS
SELECT DISTINCT *
FROM MonEquip.SALES;

-- only one column
UPDATE SALES
SET QUANTITY = ABS(TOTAL_SALES_PRICE/UNIT_SALES_PRICE)
WHERE QUANTITY <= 0;

-- check
-- quantity issue
SELECT *
from SALES
WHERE Quantity <= 0;

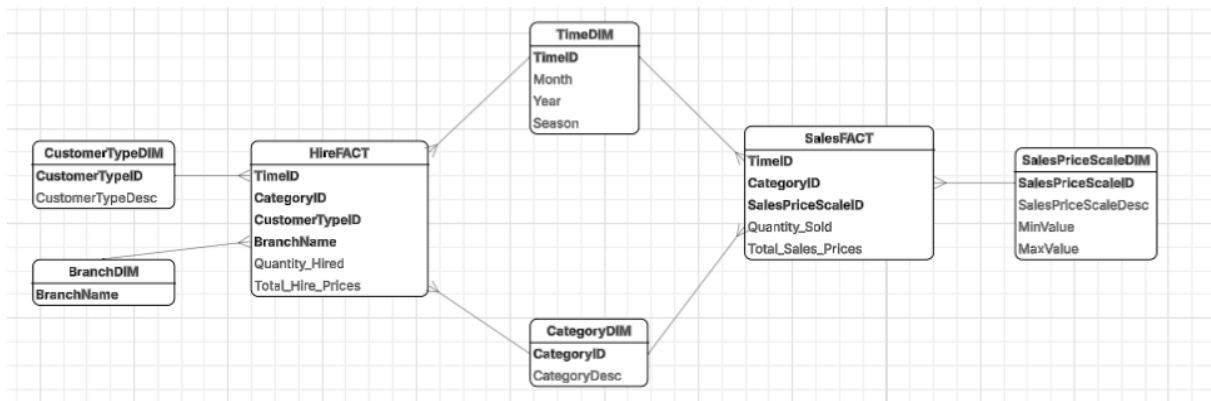
-- revenue mismatch
SELECT *
from SALES
WHERE Total_Sales_Price != Unit_Sales_Price * Quantity;

select * from SALES;

-- initial
select * from MonEquip.SALES;
```

C. Snowflake/ Star Schema (design task A)

(Highest Level of Aggregation)



F. Screenshots of table structure (design task A)

CustomerTypeDIM

Name	Null?	Type
CUSTOMERTYPEID		NUMBER
CUSTOMERTYPEDESC		VARCHAR2(20)

SQL Code

```
CREATE TABLE CustomerTypeDIM AS
SELECT CUSTOMER_TYPE_ID AS CustomerTypeID, DESCRIPTION AS
CustomerTypeDesc
FROM CUSTOMER_TYPE;
```

BranchDIM

Name	Null?	Type
BRANCHNAME		VARCHAR2(20)

SQL Code

```
CREATE TABLE BranchDIM AS
SELECT DISTINCT COMPANY_BRANCH AS BranchName
FROM STAFF;
```

HireFact

Name	Null?	Type
-----	-----	-----
TIMEID		NUMBER
CATEGORYID		NUMBER
CUSTOMERTYPEID		NUMBER
BRANCHNAME		VARCHAR2(20)
QUANTITY_HIRED		NUMBER
TOTAL_HIRE_PRICES		NUMBER

SQL Code

```
CREATE TABLE HireFACT AS
SELECT
    -- Time surrogate key (YYYYMM)
    TO_NUMBER(TO_CHAR(h.START_DATE, 'YYYYMM')) AS TimeID,

    e.CATEGORY_ID AS CategoryID,
    c.CUSTOMER_TYPE_ID AS CustomerTypeID,
    s.COMPANY_BRANCH AS BranchName,

    h.QUANTITY AS Quantity_Hired,
    h.TOTAL_HIRE_PRICE AS Total_Hire_Prices
FROM HIRE h
JOIN EQUIPMENT e ON h.EQUIPMENT_ID = e.EQUIPMENT_ID
JOIN CUSTOMER c ON h.CUSTOMER_ID = c.CUSTOMER_ID
JOIN STAFF s ON h.STAFF_ID = s.STAFF_ID;
```

TimeDIM

Name	Null?	Type
-----	-----	-----
TIMEID		NUMBER
MONTH		NUMBER
YEAR		NUMBER
SEASON		VARCHAR2(20)

SQL Code

```
CREATE TABLE TimeDIM AS
SELECT DISTINCT
    TO_NUMBER(TO_CHAR(dt, 'YYYYMM')) AS TimeID, -- surrogate key
    TO_NUMBER(TO_CHAR(dt, 'MM')) AS Month,
    TO_NUMBER(TO_CHAR(dt, 'YYYY')) AS Year
```

```

FROM (
    SELECT START_DATE AS dt FROM HIRE
    UNION
    SELECT SALES_DATE AS dt FROM SALES
);

ALTER TABLE TimeDIM ADD(
    Season VARCHAR2(20)
);

UPDATE TimeDIM
SET Season = CASE
    WHEN Month BETWEEN 9 AND 11 THEN 'Spring'
    WHEN Month IN (12, 1, 2) THEN 'Summer'
    WHEN Month BETWEEN 3 AND 5 THEN 'Autumn'
    WHEN Month BETWEEN 6 AND 8 THEN 'Winter'
END;

```

CategoryDIM

Name	Null?	Type
-----	----	-----
CATEGORYID		NUMBER
CATEGORYDESCRIPTION		VARCHAR2(20)

SQL Code

```

CREATE TABLE CategoryDIM AS
SELECT CATEGORY_ID AS CategoryID, CATEGORY_DESCRIPTION as CategoryDesc
FROM CATEGORY;

```

SalesFact

Name	Null?	Type
-----	----	-----
TIMEID		NUMBER
CATEGORYID		NUMBER
SALESPRICESCALEID		NUMBER
QUANTITY_SOLD		NUMBER
TOTAL_SALES_PRICES		NUMBER

SQL Code

```

CREATE TABLE SalesFACT AS
SELECT

```

```

-- Time surrogate key (YYYYMM)
TO_NUMBER(TO_CHAR(sal.SALES_DATE, 'YYYYMM')) AS TimeID,
e.CATEGORY_ID AS CategoryID,

-- Sales price scale (bucket by total sales price)
CASE
    WHEN sal.TOTAL_SALES_PRICE < 5000 THEN 1
    WHEN sal.TOTAL_SALES_PRICE BETWEEN 5000 AND 10000 THEN 2
    ELSE 3
END AS SalesPriceScaleID,

sal.QUANTITY AS Quantity_Sold,
sal.TOTAL_SALES_PRICE AS Total_Sales_Prices
FROM SALES sal
JOIN EQUIPMENT e ON sal.EQUIPMENT_ID = e.EQUIPMENT_ID;

```

SalesPriceScaleDIM

Name	Null?	Type
-----	-----	-----
SALESPRICESCALEID	NOT NULL	NUMBER(1)
SALESPRICESCALEDESC		VARCHAR2(20)
MINVALUE		NUMBER(7)
MAXVALUE		NUMBER(7)

SQL Code

```

CREATE TABLE SalesPriceScaleDIM (
    SalesPriceScaleID NUMBER(1) PRIMARY KEY,
    SalesPriceScaleDesc VARCHAR2(20),
    MinValue NUMBER(7),
    MaxValue NUMBER(7)
);

INSERT INTO SalesPriceScaleDIM (SalesPriceScaleID, SalesPriceScaleDesc,
MinValue, MaxValue)
VALUES (1, 'Low', 0, 4999);

INSERT INTO SalesPriceScaleDIM (SalesPriceScaleID, SalesPriceScaleDesc,
MinValue, MaxValue)
VALUES (2, 'Medium', 5000, 10000);

INSERT INTO SalesPriceScaleDIM (SalesPriceScaleID, SalesPriceScaleDesc,
MinValue, MaxValue)
VALUES (3, 'High', 10001, NULL);

```

D. List suggestion of increase the granularity of fact tables (design task B)

a. Add a new dimension

Add StaffDIM linked to both fact tables - Staff performance can be analyze - Linking it to both SalesFACT and HireFACT, allow to measure performance of each staff with indicators such as total revenue generated, number of transactions handled and everage transaction value per staff - Denormalized COMPANY_BRANCH attribute for easier branch-level staff performance analysis	STAFF_ID
	FIRST_NAME
	LAST_NAME
	GENDER
	PHONE
	EMAIL
	COMPANY_BRANCH

b. Replace an existing dimension with a higher granularity dimension

Replace CategoryDIM with EquipmentDIM - Allow to drill into each equipment level rather than just by category - Allow to analyse insights such as price trend analysis based on manaufature year or category type etc. - Enables tracking of equipment price changes (further extend using temporal dimension and SCD types)	EQUIPMENT_ID
	EQUIPMENT_NAME
	EQUIPMENT_PRICE
	MANUFACTURER
	MANUFACTURE_YEAR
	CATEGORY_TYPE
Replace CustomerTypeDIM with CustomerDIM - Drill into customers demographics (e.g., gender, location), customer behavior across different regions etc. - Allow more precise segmentation analysis	CUSTOMER_ID
	NAME
	GENDER
	ADDRESS (from STREET_NUMBER, STREET_NAME)
	SUBURB
	POSTCODE
	PHONE
	EMAIL
	CUSTOMER_TYPE
Replace TimeDIM into date granularity TimeDIM	TIME_ID

- Allows drill down from “monthly sales” to “daily sales transaction trends” such as peak transaction days, quarterly and yearly performance trends, short-term promotions impacting sales or hire trends	DAY
	MONTH
	QUARTER
	YEAR
	SEASON

E. Explanation for each SCD Types and Temporal Dimension

[An explanation of the difference among SCD types 1, 2, 3, 4, and 6.](#) Provide an example of a temporal dimension and describe which SCD type(s) would be appropriate for that dimension in Design Task A or Design Task B. If your design includes a temporal dimension, you may explain that instead.

Temporal dimension

A dimension where the record of the dimension has a life-time or period of validity (Taniar & Rahayu, 2021, ch. 6, sec. 6.3.1) which allows historical analysis of how dimension attribute changes such as price changes etc.

Each SCD Types

Type 1: Records only the most recent value of the records. When a change occurs, the old value is overwritten and no historical data is retained.

Type 2: Keeps track of the full history and the history or new record is not separated from the main dimension. A sequence number or effective date range is added to the original record to differentiate the same record in different time periods.

Type 3: Adds new column to store previous value e.g. Current Price and Previous Price column.

Type 4: Create a new separate historical dimension table to maintain the history of an attribute while the main dimension retains only the current record.

Type 6: A hybrid approach combining Type 2 and Type 3.

(Taniar & Rahayu, 2021, ch. 6)

Temporal Dimension	SCD Handling
TimeDIM (Design Task A & B)	- Does not require SCD handling because time is static and it is not a business attribute. - For example, 1st Jan 2020 will always be 1st Jan 2020. - May add future dates but never update historical rows.
EquipmentDIM (Design Task B)	The equipment price of the same type of equipment from suppliers (EQUIPMENT_PRICE) might change over time due to depreciation, supplier price updates etc.
	SCD Type 1 Overwrite the old equipment price with current equipment price however historical revenue will become inaccurate and historical prices are lost
	SCD Type 2 - Whenever the equipment price changes, insert a new record with new surrogate key (Equipment_Key) - Include effective date ranges e.g. StartDate, EndDate, and an IsCurrent flag to distinguish which version is active - This allows sales and hire transactions linked to the correct historical price ensuring the revenue analysis reflects the actual business conditions at the time of each transaction - For example, the cost of an "Excavator" purchased in 2018 was \$50,000, whereas the same equipment purchased in 2019 may cost \$55,000.
	SCD Type 3 - Add the PreviousPrice column. - Useful only when comparing current vs last price. - Stores only one previous value, which is insufficient because equipment prices may change multiple times.
	SCD Type 4 - Stores historical data in a separate table: EquipmentHistory with attributes such as EquipmentID, OldPrice, EffectiveDate, EndDate - Main table stores only the current price while history table maintains all old versions - Preserves history but makes querying more complex.
	SCD Type 6 - Combines Types 2 and 3. - Adding attributes such as - EquipmentKey (surrogate PK) - EffectiveDate, EndDate, IsCurrentFlag (Type 2) - Adding columns such as - CurrentPrice, PreviousPrice (Type 3) - Increase unnecessary complexity for MonEquip's needs.
	Conclusion Since equipment price may change when purchased again, EquipmentDIM should be managed as a temporal dimension using SCD Type 2. This ensures that hire/sales revenue reflects the correct historical equipment price at the time of the transaction.

Data Analytic Stage

G. Findings Report

Link to PowerBI dashboard:

<https://app.powerbi.com/groups/me/reports/7c0887bb-862a-4fc5-a3c2-c62150f6f027/339d4739e7707b84c905?experience=power-bi>

Get Quick Insights through PowerBI dashboard:

<https://app.powerbi.com/groups/me/insights/8dc77495-3e92-4618-a0e8-d1bda20f4386?insightsSource=Desktop&experience=power-bi>

Equipment Category Analysis

Table: Average & Sum of TOTAL_SALES_PRICES and TOTAL_HIRE_PRICES

SQL Code

```
WITH SalesAgg AS (
    SELECT
        CategoryID,
        SUM(Total_Sales_Prices) AS Total_Sales_Revenue,
        AVG(Total_Sales_Prices) AS Avg_Sales_Revenue
    FROM SalesFACT
    GROUP BY CategoryID
),
HireAgg AS (
    SELECT
        CategoryID,
        SUM(Total_Hire_Prices) AS Total_Hire_Revenue,
        AVG(Total_Hire_Prices) AS Avg_Hire_Revenue
    FROM HireFACT
    GROUP BY CategoryID
)
SELECT
    c.CategoryDesc AS Category,
    ROUND(sa.Avg_Sales_Revenue, 2) AS Avg_Sales_Revenue,
    sa.Total_Sales_Revenue,
    ROUND(ha.Avg_Hire_Revenue, 2) AS Avg_Hire_Revenue,
    ha.Total_Hire_Revenue
FROM CategoryDIM c
LEFT JOIN SalesAgg sa ON c.CategoryID = sa.CategoryID
LEFT JOIN HireAgg ha ON c.CategoryID = ha.CategoryID
ORDER BY sa.Avg_Sales_Revenue DESC;
```

- Top sales revenue category
Earthmoving generate highest total sales revenue with about 2.04M, followed by Landscaping and Air Compressor

- Highest average sales revenue per sale transaction
Earthmoving generate highest average sales revenue per sale transaction with about 156,692.31 followed by Access and Air Compressor
- Top hire revenue category
Earthmoving has the highest total hire revenue with about 36555, followed by Lighting and Rail
- Highest average total hire revenue per hire transaction
Vehicles generate highest average hire revenue per hire transaction with about 1782.5 followed by Earthmoving and Rail

Conclusion

The company's profitability is driven by sales revenue especially with equipment like earthmoving which tops the sales and hire revenue

Seasonal Trends

Pie Charts: Quantity Sold & Quantity Hired by Season

SQL Code

```
SELECT
    t.Season,
    SUM(s.Total_Sales_Prices) AS Total_Sales_Revenue,
    SUM(s.Quantity_Sold) AS Total_Quantity_Sold
FROM SalesFACT s
JOIN TimeDIM t ON s.TimeID = t.TimeID
GROUP BY t.Season
ORDER BY Total_Sales_Revenue DESC;
```

Pie Chart: Sales Revenue by Sales Price Scale & Season

SQL Code

```
SELECT
    t.Season,
    SUM(s.Total_Sales_Prices) AS Total_Sales_Revenue,
    SUM(s.Quantity_Sold) AS Total_Quantity_Sold
FROM SalesFACT s
JOIN TimeDIM t ON s.TimeID = t.TimeID
GROUP BY t.Season
ORDER BY Total_Sales_Revenue DESC;
```

- Quantity Sold
Sales peak in Spring with 32.49% while lowest in Autumn with 19.65%
- Quantity Hired
Hire peak in Spring with 32.74% while lowest in Summer with 17.02%

Conclusion

Sales and hire activities peak in Spring, possibly due to increased construction and outdoor projects. Hire revenue is lowest in Summer while sales revenue is lowest in Autumn indicating distinct seasonal demand patterns.

Branch Performance

Treemap: Hire Revenue by Branches

SQL Code

```
SELECT
    h.BranchName,
    SUM(h.Total_Hire_Prices) AS Total_Hire_Revenue
FROM HireFACT h
GROUP BY h.BranchName
ORDER BY Total_Hire_Revenue DESC;
```

- The Clayton branch leads with hire revenue of \$48K, nearly double that of Pakenham (\$33K) and slightly higher than Docklands (\$32K).
- Branches with the lowest hire revenues include Cheltenham, Dandenong, and Fitzroy.

Conclusion

Clayton and nearby branches could be regional rental hubs, while underperforming branches could benefit from localized promotions or targeted marketing to increase demand.

Sales Price Scale & Season Interaction

Bar Chart: Sales Revenue by Sales Price Scale & Season

SQL Code

```
SELECT
    sp.SalesPriceScaleDesc AS Sales_Price_Scale,
    t.Season,
    SUM(s.Total_Sales_Prices) AS Total_Sales_Revenue,
    SUM(s.Quantity_Sold) AS Total_Quantity_Sold
FROM SalesFACT s
JOIN TimeDIM t
    ON s.TimeID = t.TimeID
JOIN SalesPriceScaleDIM sp
    ON s.SalesPriceScaleID = sp.SalesPriceScaleID
GROUP BY
    sp.SalesPriceScaleDesc,
    t.Season
ORDER BY
    sp.SalesPriceScaleDesc,
    Total_Sales_Revenue DESC;
```

- High Price Scale
Generates almost all sales revenue, especially in Winter and Spring.
- Medium & Low Scales
Contribute less than 5%, showing limited lower-cost product movement.

Conclusion

MonEquip's customer base focuses on premium, high-value equipment. The company might target large construction or industrial clients rather than small contractors.

Customer Type Analysis

Pie Chart: Hire Revenue by Customer Types

SQL Code

```
SELECT
    ct.CustomerTypeDesc,
    SUM(h.Total_Hire_Prices) AS Total_Hire_Revenue,
    SUM(h.Quantity_Hired) AS Total_Quantity_Hired
FROM HireFACT h
JOIN CustomerTypeDIM ct ON h.CustomerTypeID = ct.CustomerTypeID
GROUP BY ct.CustomerTypeDesc
ORDER BY Total_Hire_Revenue DESC;
```

- Business Customers: 54.13% of Hire Revenue
- Individual Customers: 45.87% of Hire Revenue

Conclusion

Hire revenue is almost evenly split, but business customers slightly lead.

This balance indicates that both business and individual customers are important to the rental business.

Findings Summary

- Category Leader: Earthmoving - Cornerstone of sales performance
- Branch Leader: Clayton - Key revenue contributor in Hires
- Season Strength Sales and Hire Peak in Spring
- Total Hire Revenue: 317K
- Total Sales Revenue: 11.77M
- Highest Average Hire Revenue: Vehicles (1783) indicates premium rental pricing
- Highest Average Sales Revenue: Earthmoving (156,692.31)

The MonEquip business is strongly sales-driven, with the Earthmoving category and high-value equipment dominating overall revenue. While hire operations form a smaller portion, they show balance across both business and individual customers. Seasonal and regional trends (Clayton branch, Spring and Winter sales spikes) present clear opportunities for growth and marketing.

Business Recommendation

1. Strengthen Top Branches
 - Reward and expand operations at Clayton and Pakenham.
 - Review underperforming branches (Cheltenham, Dandenong, Fitzroy) for resource reallocation.
2. Expand High-Value Sales
 - Focus marketing and stock around Earthmoving, Air Compressor, and Rail equipment
3. Balance Customer Segments
 - Maintain a dual strategy — contract deals for businesses and discount offers for individual customers.
4. Leverage Seasonal Patterns
 - Run Spring sales promotions to maximize existing demand.
 - Increase Spring rental inventory for vehicles and high-demand categories.

References

Taniar, D., & Rahayu, W. (2021). *Data Warehousing and Analytics: Fueling the Data Engine*. Springer. <https://doi.org/10.1007/978-3-030-81979-8>